

**МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ**
**ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ**
**НОВОСИБИРСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ
ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ**
Факультет информационных технологий
Кафедра параллельных вычислений

ОТЧЕТ

О ВЫПОЛНЕНИИ ПРАКТИЧЕСКОЙ РАБОТЫ

**«Параллельная реализация решения системы линейных алгебраических
уравнений с помощью OpenMP»**

студента 2 курса, 24203 группы

Анисимова Льва Евгеньевича

Направление 09.03.01 – «Информатика и вычислительная техника»

Преподаватель:
доцент кафедры параллельных
вычислений
Андрей Юрьевич Власенко

Новосибирск, 2026

СОДЕРЖАНИЕ

ЦЕЛЬ.....	3
ЗАДАНИЕ.....	3
ОПИСАНИЕ РАБОТЫ.....	4
Приложение 1. Код программы <i>gen.spp</i>	6
Приложение 2. Код программы <i>tri.spp</i>	7
Приложение 3. Код скрипта запуска на кластере.....	10
Приложение 4. Таблицы.....	12
Приложение 5. Графики.....	13

ЦЕЛЬ

С помощью OpenMP распараллелить программу, реализующую итерационный метод решения СЛАУ, ознакомиться с принципами работы OpenMP, оценить эффективность параллельной программы.

ЗАДАНИЕ

Вариант: №2 — Метод минимальных невязок.

Формулировка задачи:

1. Последовательную программу из предыдущей практической работы, реализующую итерационный алгоритм решения системы линейных алгебраических уравнений вида $Ax = b$, распараллелить с помощью OpenMP.
ОБЯЗАТЕЛЬНОЕ УСЛОВИЕ: создается **одна** параллельная секция `#pragma omp parallel`, охватывающая весь итерационный алгоритм.
2. Замерить время работы программы на кластере НГУ (как минимум) на 1, 2, 4, 8, 12, 16 потоках. Построить графики зависимости времени работы программы, ускорения и эффективности распараллеливания от числа используемых ядер. Исходные данные и параметры задачи подобрать таким образом, чтобы решение задачи на одном ядре занимало не менее 30 секунд.
3. Провести исследование на определение оптимальных параметров `#pragma omp for schedule(...)` при некотором фиксированном размере задачи и количестве потоков.

ОПИСАНИЕ РАБОТЫ

В рамках выполнения лабораторной работы на языке C++ разработан и откомпилирован исходный код (Приложение 2), реализующий параллельный алгоритм решения системы линейных алгебраических уравнений (СЛАУ) методом минимальных невязок с использованием технологии OpenMP.

Экспериментальным путем подобран оптимальный размер размерности матрицы $N=2400$ и параметр точности $\epsilon = 10^{-12}$, при которых время работы последовательной программы на одном вычислительном ядре составляет не менее 30 секунд, что соответствует требованиям задания.

Параллельная реализация алгоритма основана на использовании директив OpenMP для распределения вычислительной нагрузки между потоками в рамках единого адресного пространства. Основные вычислительные блоки — умножение матрицы на вектор, вычисление норм векторов и обновление вектора решения — распараллелены с помощью директивы `#pragma omp for`.

Взаимодействие между потоками и синхронизация данных внутри итерационного цикла организованы следующим образом:

- Для вычисления глобальных скалярных произведений и норм невязки (критерий сходимости) использован `reduction`, обеспечивающий корректное агрегирование локальных сумм потоков.
- Управление жизненным циклом потоков оптимизировано путем создания единой параллельной области `#pragma omp parallel` на весь итерационный цикл, что минимизирует накладные расходы на порождение и завершение потоков.
- Обновление общих переменных (итерационных параметров) выделено в секции `#pragma omp single` с нативной барьерной синхронизацией.

Проведены замеры времени выполнения программы при использовании 1, 2, 4, 8, 12, 16, 20 и 24 вычислительных потоков на узле кластера. В конфигурациях свыше 12 потоков исследовано влияние технологии *Hyper-Threading* на общую производительность.

Дополнительно проведено исследование зависимости времени работы от параметров планировщика в циклах (директива `schedule`). Выполнен сравнительный анализ стратегий `static` и `dynamic` с размерами порций (`chunk`) {1, 10, 100}. Установлено, что для данной вычислительной задачи наиболее эффективным является режим `static` с большим размером порции, что обусловлено лучшей локальностью данных в кэш-памяти процессора.

На основании полученных данных выполнен расчет показателей ускорения и эффективности распараллеливания. Результаты сведены в таблицу (Приложение 3), а визуализация зависимостей представлена в виде графиков (Приложение 4).

ЗАКЛЮЧЕНИЕ

В ходе выполнения лабораторной работы была реализована параллельная версия метода минимальных невязок для решения СЛАУ с использованием технологии OpenMP. Экспериментальные данные подтвердили, что использование многопоточности обеспечивает существенное ускорение работы программы до определенного предела (8–12 потоков). Однако детальный анализ производительности показал, что итерационный процесс сильно ограничен пропускной способностью шины памяти, что характерно для матрично-векторных операций.

Основной вклад в накладные расходы вносит конкуренция потоков за доступ к общим данным в оперативной памяти и задержки, возникающие при синхронизации вычислительных фаз. Исследование различных стратегий планирования (Schedule) выявило, что при малых размерах порции данных (chunk=1) накладные расходы на управление циклом и эффекты «ложного разделения» данных в кэш-линиях значительно снижают производительность. Использование параметра static с размером порции 100 позволило оптимизировать использование кэш-памяти L2/L3 и сократить время выполнения на 30–40% по сравнению с динамическим распределением.

Особое внимание заслуживает поведение программы при использовании более 12 потоков (режим Hyper-Threading). На построенных графиках отчетливо визуализируется деградация времени выполнения при переходе к 16, 20 и 24 потокам. Это обусловлено тем, что логические ядра делят между собой одни и те же исполнительные блоки и ресурсы кэша, что в задачах с высокой интенсивностью работы с памятью приводит к росту задержек и падению эффективности до 11–18%.

На основании полученных зависимостей времени, ускорения и эффективности можно сделать вывод о целесообразности использования данной параллельной реализации на конфигурациях, не превышающих число физических ядер процессора (до 12 потоков). Дальнейшее масштабирование ограничивается «стеной памяти» (Memory Wall), что подтверждает классическую зависимость снижения параллельной эффективности для алгоритмов с низким отношением числа арифметических операций к объему передаваемых данных из памяти.

Приложение 1. Код программы gen.cpp

```
#include <vector>
#include <random>
#include <fstream>
#include <iostream>
#include <string>

using namespace std;

int main() {
    int n = 2400;
    string mat_file = "matrix.bin";
    string vec_file = "vector_b.bin";

    mt19937 gen(42);
    uniform_real_distribution<double> dist(-1.0, 1.0);
    uniform_real_distribution<double> dist_rhs(-1.0, 1.0);

    vector<double> R(n * n);
    vector<double> A(n * n, 0.0);
    vector<double> b(n);

    cout << "Генерация системы " << n << "x" << n << " (улучшенная
    версия)..." << endl;

    // 1. Генерируем вспомогательную матрицу R
    for (auto &v : R) v = dist(gen);

    // 2.  $A = R^T * R + 0.5 * I$ 
    for (int i = 0; i < n; ++i) {
        for (int j = 0; j < n; ++j) {
            for (int k = 0; k < n; ++k) {
                A[i * n + j] += R[k * n + i] * R[k * n + j];
            }
        }
        A[i * n + i] += 10.0; // eps = 0.5
    }

    // 3. b
    for (auto &v : b) v = dist_rhs(gen);

    ofstream f_mat(mat_file, ios::binary);
    f_mat.write(reinterpret_cast<char*>(A.data()), A.size() *
    sizeof(double));
    f_mat.close();

    ofstream f_vec(vec_file, ios::binary);
    f_vec.write(reinterpret_cast<char*>(b.data()), b.size() *
    sizeof(double));
    f_vec.close();

    cout << "Файлы готовы: " << mat_file << ", " << vec_file << endl;
    return 0;
}
```

Приложение 2. Код программы *tri.cpp*

```
#include <iostream>
#include <vector>
#include <fstream>
#include <cmath>
#include <iomanip>
#include <omp.h>

constexpr int N = 2400;
constexpr double EPS = 1e-12;
constexpr int MAX_ITER = 15000;

int main() {
    std::vector<double> A(N * N);
    std::vector<double> b(N);
    std::vector<double> x(N, 0.0);
    std::vector<double> r(N), Ar(N);

    std::ifstream fa("matrix.bin", std::ios::binary);
    if (!fa) return 1;
    fa.read((char*)A.data(), N * N * sizeof(double));
    std::ifstream fb("vector_b.bin", std::ios::binary);
    if (!fb) return 1;
    fb.read((char*)b.data(), N * sizeof(double));

    double b_norm2 = 0.0;
    for (double val : b) b_norm2 += val * val;
    double r0_norm = std::sqrt(b_norm2);

    int converged_iter = -1;
    double r_norm2_shared = 0.0;
    double ArAr_shared = 0.0;
    double rAr_shared = 0.0;
    double tau = 0.0;
    bool stop_flag = false;

    double t_start = omp_get_wtime();

    #pragma omp parallel shared(A, b, x, r, Ar, stop_flag, converged_iter,
tau)
    {
        for (int iter = 0; iter < MAX_ITER; ++iter) {

            // 1. r = b - Ax
            #pragma omp for schedule(runtime)
            for (int i = 0; i < N; ++i) {
                double Ax_i = 0.0;
                for (int j = 0; j < N; ++j) {
                    Ax_i += A[i * N + j] * x[j];
                }
                r[i] = b[i] - Ax_i;
            }

            // 2. Норма невязки
            #pragma omp single
            r_norm2_shared = 0.0;

            #pragma omp for reduction(+:r_norm2_shared) schedule(runtime)
            for (int i = 0; i < N; ++i) {
                r_norm2_shared += r[i] * r[i];
            }

            #pragma omp single
```

```

        {
            if (std::sqrt(r_norm2_shared) / r0_norm < EPS) {
                converged_iter = iter;
                stop_flag = true;
            }
        }

// Проверка флага остановки всеми потоками
if (stop_flag) break;

// 3. Ar = A * r
#pragma omp for schedule(runtime)
for (int i = 0; i < N; ++i) {
    double Ar_i = 0.0;
    for (int j = 0; j < N; ++j) {
        Ar_i += A[i * N + j] * r[j];
    }
    Ar[i] = Ar_i;
}

// 4. Вычисление tau
#pragma omp single
{ rAr_shared = 0.0; ArAr_shared = 0.0; }

        #pragma omp for reduction(+:rAr_shared, ArAr_shared)
schedule(runtime)
    for (int i = 0; i < N; ++i) {
        rAr_shared += r[i] * Ar[i];
        ArAr_shared += Ar[i] * Ar[i];
    }

    #pragma omp single
    tau = rAr_shared / ArAr_shared;

// 5. x = x + tau * r
#pragma omp for schedule(runtime)
for (int i = 0; i < N; ++i) {
    x[i] += tau * r[i];
}
}

double t_end = omp_get_wtime();
double final_residual = std::sqrt(r_norm2_shared) / r0_norm;

std::cout << "=====" << std::endl;
std::cout << "OpenMP Statistics (No Manual AVX):" << std::endl;
std::cout << "    Time taken:      " << std::fixed << std::setprecision(4)
<< (t_end - t_start) << " s" << std::endl;
std::cout << "    Threads:          " << omp_get_max_threads() << std::endl;
std::cout << "-----" << std::endl;
std::cout << "Algorithm Results:" << std::endl;
if (converged_iter != -1) {
    std::cout << "    Status:          CONVERGED" << std::endl;
    std::cout << "    Iterations:      " << converged_iter << std::endl;
} else {
    std::cout << "    Status:          DID NOT CONVERGE" << std::endl;
    std::cout << "    Iterations:      " << MAX_ITER << std::endl;
}
std::cout << "    Final Residual: " << std::scientific << final_residual <<
std::endl;
std::cout << "    x[0]:            " << std::fixed << std::setprecision(6)
<< x[0] << std::endl;
std::cout << "    x[N-1]:          " << std::fixed << std::setprecision(6)

```



```
<< x[N-1] << std::endl;
    std::cout << "======" << std::endl;

    return 0;
}
```

Приложение 3. Код скрипта запуска на кластере

Для прогона chunk:

```
#!/bin/bash
#PBS -N SLAU_SCHEDULE_TEST
#PBS -l nodes=1:ppn=12
#PBS -l walltime=00:30:00
#PBS -j oe

cd $PBS_O_WORKDIR

# Компилируем один раз
g++ -O3 -fopenmp -march=native ../src/omp.cpp -o schedule_prog

RESULT_FILE="SCHEDULE_RESULT.txt"
echo "=== SCHEDULE INVESTIGATION: $N=2400, Threads=12 ===" > $RESULT_FILE

# Фиксируем потоки
export OMP_NUM_THREADS=12

SCHED_TYPES="static dynamic"
CHUNK_SIZES="1 10 100"

for type in $SCHED_TYPES
do
    for chunk in $CHUNK_SIZES
    do
        echo "Testing $type with chunk $chunk..."
        export OMP_SCHEDULE="$type,$chunk"

        echo -e "\n[SCHEDULE: $type, $chunk]" >> $RESULT_FILE
        # Запускаем и пишем только нужные данные
        ./schedule_prog >> $RESULT_FILE
    done
done

echo "Done. Check $RESULT_FILE"
```

Для сравнения по потокам:

```
#!/bin/bash
#PBS -N SLAU_FULL_REPORT
#PBS -l nodes=1:ppn=12
#PBS -l walltime=00:40:00
#PBS -j oe

cd $PBS_O_WORKDIR
g++ -O3 -fopenmp gen.cpp -o generate
g++ -O3 -fopenmp -march=native ../src/omp.cpp -o omp_prog

RESULT_FILE="RESULT.txt"
CSV_FILE="bench_results.csv"

echo "=== BENCHMARK START: $(date) ===" > $RESULT_FILE
echo "CPU Info: $(lscpu | grep 'Model name' | cut -d: -f2 | xargs)" >> $RESULT_FILE
echo "Threads,Time" > $CSV_FILE

echo "Generating matrix..." >> $RESULT_FILE
./generate >> $RESULT_FILE 2>&1
```

```

echo "=== Запуск замеров производительности ===" >> $RESULT_FILE

THREADS_LIST="1 2 4 8 12 16 20 24"

for t in $THREADS_LIST
do
    echo "Running with $t threads..."

    echo -e "\n>>> TEST RUN: THREADS = $t <<<" >> $RESULT_FILE
    echo "-----" >> $RESULT_FILE

    export OMP_NUM_THREADS=$t
    export OMP_PROC_BIND=false

    OUTPUT=$(./omp_prog)
    echo "$OUTPUT" >> $RESULT_FILE

    TIME=$(echo "$OUTPUT" | grep "Time:" | awk '{print $2}' | sed 's/s//')
    echo "$t,$TIME" >> $CSV_FILE

    echo "-----" >> $RESULT_FILE
    echo "Done for $t threads. Time: $TIME s"
done

echo -e "\n=== Замеры завершены. Сводка в $CSV_FILE ===" >> $RESULT_FILE

```

Приложение 4. Таблицы

	ВРЕМЯ	УСКОРЕНИЕ	ЭФФЕКТИВНОСТЬ	ВРЕМЯ ПОСЛЕДОВАТЕЛЬНОЙ: 49.9033
2	25.7168	1.940	97.02%	
4	16.4863	3.027	75.67%	
8	14.0333	3.556	44.45%	
12	14.0848	3.543	29.52%	
16	17.1780	2.905	18.16%	
20	17.4084	2.866	14.33%	
24	18.3041	2.726	11.36%	

Приложение 5. Графики

